

สรุปการเปลี่ยนแปลงระบบ ETL (SDOQAP)

branch `restructure/monorepo` · ฐาน `bell` · อ้างอิง tag `report-snapshot-2026-09-30` · 34 commits

3,044 →
1,966

บรรทัดของ
spark_quality_engine.py

21

stage ที่มีชื่อ เทสต์ และจับเวลา
แยก

100%

recall ทั้ง 4 ชนิดปัญหา
(precision 95.9%)

1M แกว

ประมวลผลสำเร็จ 414 วินาที

1. ภาพรวม: ทำอะไรไปบ้าง

งานแบ่งเป็น 4 แผนตามผลรีวิรวระบบ ETL และเกณฑ์การให้คะแนน (CLO5 55% และ CLO6 15%) โดยไม่แตะระบบ Dashboard (หน้า Dashboards/Query & Metrics, `analytics.py`, Gold layer, Grafana, Kibana — ย้ายโฟลเดอร์ได้แต่ไม่แก้เนื้อหา)

แผน	สิ่งที่เปลี่ยน	เกณฑ์ที่รองรับ
A โครงสร้าง โฟลเดอร์	ย้ายเป็น monorepo มาตรฐาน: <code>services/</code> (<code>api</code> , <code>spark</code> , <code>ui</code>), <code>infra/</code> , <code>data/</code> , <code>scripts/</code> , <code>docs/</code> ลบไฟล์ซ้ำและโค้ดที่ไม่มีใครใช้ เพิ่ม <code>.env.example</code> และให้ CI รันเทสต์จริง	ระเบียบโครงการ
B Extraction + Loading	แต่ละการนำเข้าได้ <code>ingest_id</code> และโฟลเดอร์ <code>raw</code> ของตัวเอง, ตารางสถานะรอบ <code>sdoqap_runs</code> , คิวต่อตาราง, กันไฟล์ซ้ำด้วย checksum, เขียน quarantine แบบรันซ้ำได้ก่อน MERGE, ย้าย <code>raw</code> ไป archive, ปิดช่องโหว่ความปลอดภัยของจุดนำเข้า	Extraction (10), Loading (5)
C Transformation	แยก <code>run_quality_check</code> (~1,200 บรรทัด) เป็น 21 stage, รวมตัวคำนวณ similarity 4 ชุดเหลือชุดเดียว, เพิ่มกฎช่วงค่าตามธุรกิจ (score 0–100), ให้ Elasticsearch เป็นแหล่งความจริงเดียวของกฎและ schema	จำนวนกระบวนการ (15), Transformation (10)
D หลักฐานตาม เกณฑ์	สคริปต์วัด detection เทียบ ground truth, benchmark 10K–1M แกว, รายงานการใช้ประโยชน์ของข้อมูล, สำรองแหล่งข้อมูล และตัวสร้างรายงาน <code>rubric-mapping.md</code> จากผลรันจริง	ทุกหัวข้อ รวม CLO6

2. ปัญหาที่พบในรีวิว และวิธีแก้

ID	ปัญหา	วิธีแก้
F-D1	อัปโหลดซ้อนกันเขียนทับไฟล์ <code>raw</code> เดิม และ Spark ลบทั้งโฟลเดอร์เมื่อจบ → ข้อมูลหาย	โฟลเดอร์ <code>/data/raw/<table>/<ingest_id>/</code> ต่อการนำเข้าหนึ่งครั้ง, archive แทนการลบ, คิว FIFO ต่อตาราง
F-D3	API ตอบ "triggered" แม้ Spark ไม่ได้รัน (fallback สั่ง Popen ใน container ที่ไม่มี pyspark)	ลบ fallback ออก ถ้าเรียก daemon ไม่ได้ตอบ 503 และบันทึกสถานะ <code>TRIGGER_FAILED</code>
F-S1	Trigger daemon เปิดพอร์ต 8099 ออก host ไม่มี auth	ปิดพอร์ตจาก host, เพิ่ม shared secret (<code>X-Trigger-Secret</code>) และตรวจสอบชื่อตาราง

F-S2/S3	allowlist URL เป็น fail-open และตาม redirect ได้; เช็ค SELECT ดูแค่ค่าแรก	fail-closed, ตรวจสอบทุก redirect hop, รัน SQL ใน transaction read-only พร้อม timeout และเพดานจำนวนแถว
F-S4	รหัสผ่าน Elasticsearch/pgAdmin ผังใน compose และโค้ด	ต้องมาจาก <code>.env</code> (<code>\${VAR:?}</code>) ส่วน Kibana/Grafana ไม่แต่ตามขอบเขต
F-D4/D5	เขียน active ก่อน quarantine ไม่ atomic; lock หมดอายุ 15 นาทีไม่มี heartbeat	quarantine ก่อน MERGE (ลบตาม ingest_id แล้วเขียนใหม่), heartbeat ต่ออายุ lock ทุก 60 วินาที
F-D6	lock ไม่ว่าง → exit 0 → daemon เริ่ม remediation จากผลรอบเก่า	exit code เฉพาะ (75 = SKIPPED) และ daemon อ่านผลตาม <code>ingest_id</code> ของตัวเอง
F-D7	connector จำลองคืนจำนวนแถวปลอม 10,100	ติดป้าย "โหมดสาริต" และไม่คืนตัวเลขที่ไม่ได้วัด
F-M1-M4	<code>run_quality_check</code> ยาวเกิน, similarity ซ้ำ 4 ชุด, helper ซ้ำ, กฎ/schema มีสองแหล่ง	21 stage ใน <code>sdoqap/</code> , <code>SemanticMatcher</code> ชุดเดียว, <code>sdoqap.common</code> , ES เป็นแหล่งเดียว (ไฟล์ JSON เป็น seed)
F-T1/T2	CI ไม่รันเทสต์; เทสต์ของ Spark ถูกตัดออก	CI มี 3 job ใหม่: API pytest, UI vitest, Spark unit tests

3. โครงสร้างโฟลเดอร์ ก่อน → หลัง

ก่อน (แบบที่ root)

```
api/ spark/ ui/ tests/
nginx/ grafana/ prometheus/ n8n/
airflow/ (ไม่มี service ใช้)
sample_data/ dummy_data/ stress test/
user_inputs/
*.xlsx *.csv *.txt *.md (ปนที่ root)
student_course_score_evaluation_dataset*
(2 ที่ ซ้ำกัน)
scripts/ (ปนกับสคริปต์ซ้ำของ spark)
```

หลัง (monorepo)

```
services/ api/ spark/ ui/
infra/ nginx/ grafana/ prometheus/
n8n/ swarm/
data/ evaluation/ samples/
stress/ inputs/
scripts/ ops/ windows/ evaluation/
dev/ maintenance/
docs/ architecture/ requirements/
specs/ reports/ evaluation/
root: README, docker-compose.yml,
.env.example, *.bat
```

ย้ายเฉพาะ path ผัง host — path ภายใน container (`/app`, `/opt/spark-apps` ฯลฯ) คงเดิม รายงานเก่าที่อ้าง `path:line` ตรวจสอบได้ด้วย `check_citations.py --ref report-snapshot-2026-09-30`

4. Activity ของเส้นทาง ETL หลังแก้

ผู้ใช้/API	ตรวจสอบสิทธิ์ → ตรวจสอบชื่อตาราง → อ่านข้อมูล (จำกัดขนาด/allowlist/read-only) → checksum ซ้ำ? (ตอบ duplicate) → ตรวจสอบคีย์หลักตาม registry → เขียน HDFS /data/raw/<t>/<ingest_id>/ → สร้างสถานะ QUEUED
Trigger daemon	ตรวจสอบ secret → คิวต่อตาราง (FIFO) → spark-submit --ingest-id
Spark (21 stage)	lock+heartbeat → schema_align → schema_drift → auto_clean → validation → dedup → standardize → range_rules → anomaly (IQR/Z/induced) → quarantine_assembly → column_filter → เขียน quarantine (idempotent) → Delta MERGE active → distribution → breakdown → copdq → freshness → quality_score → ai_advisory → operational_impact → report (ES)
หลังรัน	archive raw → (ถ้ามี quarantine) auto-remediation + ตรวจสอบซ้ำ → ตั้งสถานะสุดท้ายหนึ่งครั้ง: SUCCEEDED / FAILED / SKIPPED

5. ผลการวัดจากระบบจริง (ข้อมูลประเมิน 10,100 แถว)

5.1 Detection เทียบ ground truth

ชนิดปัญหา	จริง	ตรวจพบ	อัตรา
Missing Score	300	300	100%
Invalid Score Range	200	200	100%
Study Hours Outlier	100	100	100%
Duplicate	100	100	100%

การตั้งค่า IQR	precision	recall	accuracy	false positive
ตัวคูณ 1.5	0.874	1.000	0.990	101
ตัวคูณ 3.0 (ใช้จริง)	0.959	1.000	0.997	30

ที่ 1.5 ระบบจับนักศึกษาปกติที่เรียน 10–12 ชั่วโมงและคะแนน 38–51 เป็นความผิดปกติ ค่า IQR 3.0 เป็นค่าของ test case นี้เท่านั้น ไม่ใช่ค่าที่ถูกต้องสำหรับข้อมูลทุกประเภท แถวซ้ำ 100 แถวถูก `auto_clean` ตัดก่อนนับ (นับเป็น dropped ไม่อยู่ใน quarantine)

5.2 เวลาประมวลผลตามขนาดข้อมูล

แถว	สถานะ	เวลาใน engine (วินาที)	end-to-end (วินาที)
10,000	SUCCEEDED	66.2	95
100,000	SUCCEEDED	85.5	116
500,000	SUCCEEDED	203.4	238
1,000,000	SUCCEEDED	414.2	454

5.3 การนำข้อมูลไปใช้ และการพิสูจน์ว่า refactor ไม่เปลี่ยนผล

- ข้อมูลสะอาด 9,370 แถว → คะแนนเฉลี่ยรายวิชา, อัตราผ่าน 99.95%, การกระจายคะแนน, นักศึกษาที่ต้องติดตาม 5 คน (docs/evaluation/d-utilization.md)
- Golden test ก่อน/หลังแยก stage: ตรงกันทุกตัวเลข (250 / 227 / 23 แถว และ breakdown ของ quarantine)

- e2e: อัปโหลด 2 ไฟล์ติดกัน → ทั้งคู่ถูกประมวลผลต่อคิว ไม่ทับกัน, ไฟล์ซ้ำถูกตรวจพบ, raw ถูก archive

6. การทดสอบ

ชุด	จำนวน	รันที่
API (pytest)	52	ใน container <code>api</code>
Spark unit (21 stage, similarity, daemon, seed)	60	ใน container <code>spark-master</code>
UI (vitest)	76	host (<code>npm test</code>)
สคริปต์ประเมิน + ตัวตรวจ citation	21 + 10	host

7. สิ่งที่ทำต่างจากแผน และข้อจำกัดที่ทราบ

ต้องรู้ก่อนนำไปใช้

- การนำเข้า HDFS/Spark ไม่มีฟอร์มในหน้า UI: พังก์ชัน `handleCsvSubmit/handleApiSubmit/handleRdbmsSubmit` ในหน้า Ingestion ไม่ถูกเรียกเลย ฟอร์มทุกแท็บเรียกแค่เอนจินได้ตอบ จึงสร้าง hook/component แสดงสถานะรอบ (`RunStatusLine`) ไว้แต่ยังไม่ต่อเข้าหน้านี้
- สถานะ **SUCCEEDED** มาเร็วเกินไปในร่างแรก: ตรวจพบตอน e2e แล้วแก้ไข daemon ตั้งสถานะสุดท้ายหนึ่งครั้งหลังจบ remediation และการตรวจซ้ำ
- ข้อความ **log** ของ **daemon** ต้องคงเดิม: หน้า Ingestion อ่านสถานะจากข้อความใน log จึงคงถ้อยคำเดิมไว้
- ค่า **IQR**: ตั้ง 3.0 ตามแผน (ชั่วคราวเคยคงไว้ 1.5) และเก็บผลเปรียบเทียบทั้งสองค่าเป็นหลักฐาน
- เอนจินได้ตอบ: ประเมินใน workspace ชั่วคราว เพื่อไม่ทับชุดข้อมูลและ state ที่ API ใช้อยู่ เอนจินนี้ยังใช้สถานะชุดเดียวร่วมกันทุกผู้ใช้
- เทสต์ **API 1** ข้อแก้ค่าที่คาด: `test_multi_table_preview` จาก 10,000 เป็น 9,980 ตาม generator ที่เว้นทุก 500 คน ส่วนไฟล์ `dirty_dataset.csv` ที่ API ใช้อยู่เป็นชุดตัวอย่าง 1,030 แถว ต้อง mount ชุดต้นฉบับบนตอนรันเทสต์
- ยังไม่ได้ตรวจบน **GitHub Actions**: job ใหม่ทั้ง 3 ตัวรันได้ในเครื่องเท่านั้น; `ollama` ขึ้น unhealthy ตามเดิมเพราะ image ไม่มี `curl`
- CLO6 (15 คะแนน)** รายงานเป็นคำอธิบายนวัตกรรมที่มีอยู่แล้ว ยังไม่มีการวัดว่านวัตกรรมแต่ละตัวช่วยได้กี่ %

8. วิธีทดสอบซ้ำ (Git Bash ที่ C:\ETL)

<code>docker compose up -d</code>	# รอ ~3 นาที
<code>bash scripts/ops/e2e_ingest_check.sh</code>	# Extraction
<code>bash scripts/evaluation/run_batch_evaluation.sh</code>	# Transformation + ประโยชน์ข้อมูล
<code>python scripts/evaluation/run_scale_benchmark.py</code>	# Loading ตามขนาด (รวม ~15 นาที)
<code>python scripts/evaluation/build_rubric_report.py</code>	# สรุป docs/evaluation/rubric-mapping.md

รายงานเดิม: docs/evaluation/rubric-mapping.md · รีวิวต้นทาง: docs/etl-review/2026-09-30-etl-system-review.md